CS 4240: Compilers & Interpreters 09/24/2025

Homework 2: Instruction Scheduling & Register Allocation

Professor Qirun Zhang Due: 10/15/2025 11:59pm

- **We recommend you type your solutions using Overleaf or any other software. Pictures of hand written solutions will be accepted.**
- **All homework must be done independently. You may post clarification questions on Piazza, but be sure to make your post instructor-only if it includes part of your solution.**
- **Late submissions will be accepted within 24 hours of the assignment due date and will receive a 50% point deduction. Any assignments submitted more than 24 hours late will receive a 100% point deduction.**

1 Instruction Selection

(35 points) Consider the following sequence of IR instructions generated from the source code program fragment, "`a[b[i]] = a[j+1]`", for arrays with 4-byte elements (e.g., arrays of int's):

```
1.  t1 := i*4 // Offset calculation
2.  t2 := b+t1 // Address calculation { address in t2
3.  t3 := *t2 // Load the value at address t2 into t3
4.  t4 := t3*4 // Offset calculation
5.  t5 := a+t4 // Calculate address for store in t5
6.  t6 := j+1 // Load index
7.  t7 := t6*4 // Offset calculation
8.  t8 := a+t7 // Address calculation { address in t8
9.  t9 := *t8 // Load the value at address t8 into t9
10. *t5 := t9 // Store value t8 into location with address t5
```

The ISA that we are targeting for this problem includes the following instructions, with costs. This problem is focused on instruction selection. Assume that register allocation will be performed in a later pass, so you can generate instructions that refer to virtual IR registers such as i, j, a, b, and t1...t9

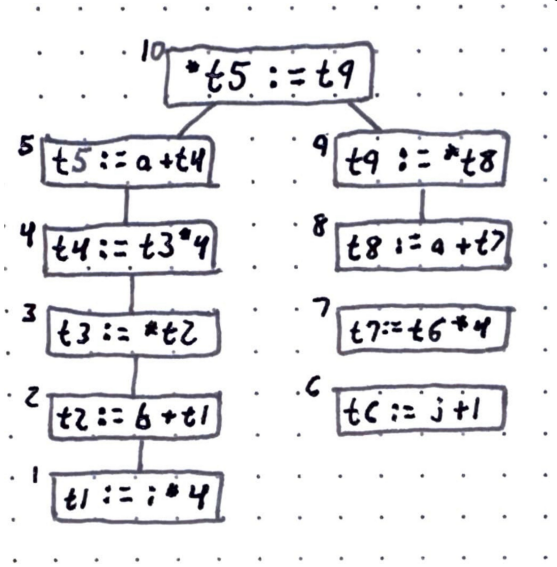
Instruction	Operation	Comment	Cost
`add r3, r2, r1`	r1 ← r2 + r3		2
`addi c, r2, r1`	r1 ← r2 + c		1
`mul r3, r2, r1`	r1 ← r2 × r3		4
`muli c, r2, r1`	r1 ← r2 × c		4
`lshi c, r2, r1`	r1 ← r2 << c	`// Left shift r2's content by c bits` `// (each shift equals multiplication by 2)`	1
`lw r2, r1`	r1 ← *r2	`// Load value from address contained in r2`	7
`lwo r3, r2, r1`	r1 ← *(r2 + 4×r3)	`// Do offset calculation with word offset` `// and load value from the computed address`	7
`sw r2, r1`	*r1 ← r2	`// Store r2 into address contained in r1`	4
`mm r2, r1`	*r1 ← *r2	`// Move (copy) value from address` `// in r2 to that in r1`	9
`mmo r3, r2, r1`	*r1 ← *(r2+r3)	`// Do offset calculation and then move value`	11
`mmc r3, r2, r1`	*(r1 + r3) ← *(r2+r3)	`// Memory to memory move, r1 and r2` `// are base pointers r3 is the same offset`	12

(a.) (10 points) Show the output of a simple instruction selection pass that emits machine instructions for one IR instruction at a time. The output should minimize the cost for each instruction. Include the sequence of machine instructions with virtual registers, as well as the total cost.

```
1.  t1 := i*4          → lshi 2, i, t1          // cost 1
2.  t2 := b+t1         → add  t1, b, t2         // cost 2
3.  t3 := *t2          → lw   t2, t3            // cost 7
4.  t4 := t3*4         → lshi 2, t3, t4         // cost 1
5.  t5 := a+t4         → add  t4, a, t5         // cost 2
6.  t6 := j+1          → addi 1, j, t6          // cost 1
7.  t7 := t6*4         → lshi 2, t6, t7         // cost 1
8.  t8 := a+t7         → add  t7, a, t8         // cost 2
9.  t9 := *t8          → lw   t8, t9            // cost 7
10. *t5 := t9          → sw   t9, t5            // cost 4
```

The total cost of all instructions is 28.

(b.) (5 points) Show the dependence tree for the IR instructions, in which each vertex represents an IR instruction and each edge represents a dependence from a def to a use.



(c.) (10 points) Perform tree-based tiling using the greedy technique working top down to generate complex instructions that may subsume multiple IR operations and show the output of the resulting instruction selection pass. Compare the cost with that of part a) above.

```
lwo  i, b, t3          // cost 7
lshi 2, t3, t4         // cost 1
add  t4, a, t5         // cost 2
addi 1, j, t6          // cost 1
lshi 2, t6, t7         // cost 1
add  t7, a, t8         // cost 2
mm   t8, t5           // cost 9
```

The total cost of all instructions is 23, which is 5 less than that of part (a).

(d.) (10 points) Perform tree-based tiling using the optimal dynamic programming technique, and show the output of the resulting instruction selection pass. Compare the cost with that of parts a) and c) above. You can write programs to solve this task, but you don't need to include your program's code.

```
lwo  i, b, t3     // cost 7 (start of left tree... dest addr)
lshi 2, t3, t4     // cost 1
add  t4, a, t5     // cost 2 (end left)
addi 1, j, t6     // cost 1 (start of right tree... value)
lwo  t6, a, t9     // cost 7 (end right)
sw,  t9, t5         // cost 4 (head)
```

This approach has a total cost of 22, which is one less than that of part (c) and 6 less than that of part (a). Of all of the costs, this is most optimal in terms of cost.

2 Register Allocation

(35 points) Consider the following implementation, `fact`, of the factorial function in an IR, which takes an argument in virtual register n and returns a value stored in `acc`:

```
1. start:                  // label for start of program
2.      acc := 1
3.      ctr := n
4. loop:                   // label for loop entry
5.      con := ctr <= 1
6.      br con, done       // branch to label done if c = true
7.      acc := acc * ctr
8.      ctr := ctr - 1
9.      jmp loop           // branch to label loop
10. done:                  // label for loop exit
11.     return acc         // return acc
```

(a.) (15 points) Show the live ranges for each local variable (symbolic register) that occurs in fact. LIVE(v), the live range for variable v, is the set of program points at which variable v is live. As discussed in class, please use the i- and i+ notation to denote program points just before and just after instruction i. For example, variable acc is not live at 2-, but is live at 2+ which is equivalent to 3-. Thus, LIVE(acc) should include 2+ or 3-, but not 2-.

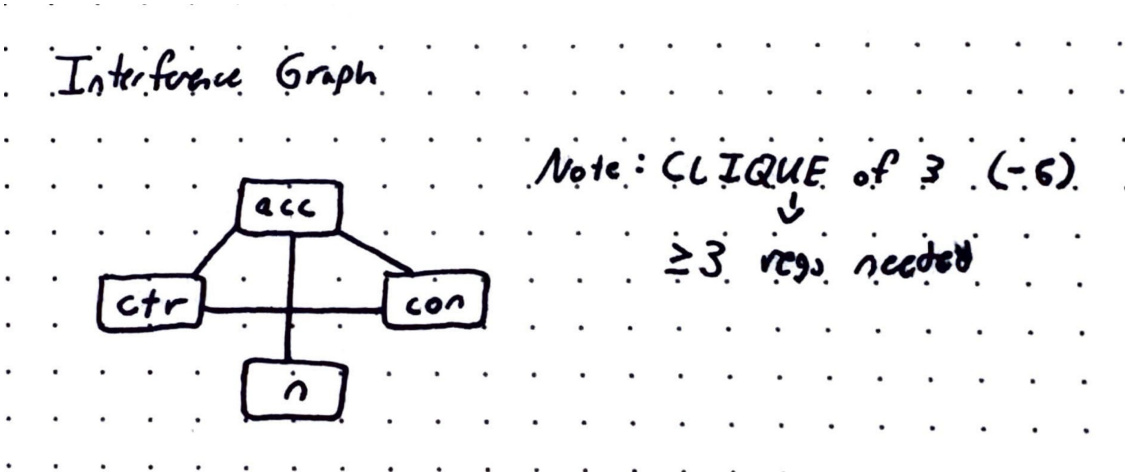
```
// use only to initalize at (3)
LIVE(n)={1+, 2-, 2+, 3-}

// use everywhere (body and ret)
LIVE(acc)={2+, 3-, 3+, 4-, 5-, 5+, 6-, 6+, 7-, 7+,
           8-, 8+, 9-, 9+, 10-, 10+, 11-}

// loop... dead at 'done'
LIVE(ctr)={3+, 4-, 5-, 5+, 6-, 6+, 7-, 7+, 8-, 8+, 9-, 9+}

// define at (5); only used at (6)
LIVE(con)={5+, 6-}
```

(b.) (10 points) Show the interference graph for this program, in which each vertex corresponds to a local variable, and an undirected edge between two variables indicates that their LIVE sets have a non-empty intersection.



(c.) (10 points) Show the output of a spill-free register allocation for the above program, using the minimum number of physical registers. Your allocation should not assign the same register to more than one live variable at any program point.

```
1. start:
2.      R1 := 1  // let R1 be for acc
3.      // we removed this line via optimization
4. loop:
5.      R3 := R2 <= 1   // let R3 be con
6.      br R3, done
7.      R1 := R1 * R2   // ac and ctr (R2) logic
8.      R2 := R2 - 1
9.      jmp loop
10. done:
11.     return R1   // acc
```

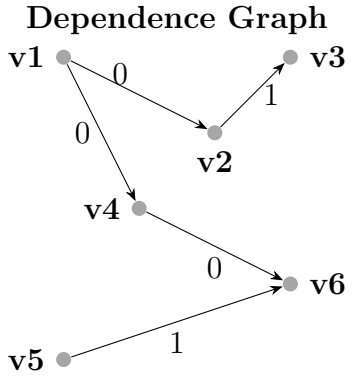
NOTE: as per our note in part (b), we observe we need 3 or more registers. Since this is using minimal registers, we use three.

3 Instruction Scheduling and Register Allocation

(30 points) Consider the following sequence of assembly code instructions generated from C-like pseudocode of the form, `Z = X[i]; print &X[i+2+(*N)];`" for a hypothetical RISC processor with machine instructions that are similar to IR instructions. For convenience, we use temporaries, t1, t2, t4, t5, and t6 to denote virtual/symbolic registers, and omit instructions for the print operation which will read the value of `&X[i+2+(*N)]` from 6. We also assume in this problem that no registers need to be allocated for integer i and addresses X, Z, and N, because they may already be in registers as procedure arguments or they may be known constants.

```
1. t1 := X + i      // Address of X[i] for single-byte elements
2. t2 := *t1        // Load the value at address X[i] into t2
3. *Z := t2         // Store the value of t2 into address Z
4. t4 := t1 + 2
5. t5 := *N         // Load the value at address N into t5
6. t6 := t4 + t5    // Compute address of X[i+2+(*N)] in t6
```

The dependence graph for this machine code is shown below for instructions with virtual registers, where vertex vi corresponds to instruction i. Each outgoing edge from a load instruction (e.g., from v2 or v5) has a latency of 1 to indicate an extra cycle needed to complete the load, beyond the unit cycle needed for each instruction. For example, this implies that the start time of instruction v3 must be at least 2 cycles after the start time of instruction v2 in any legal schedule of this dependence graph.



In the following questions, you only need to show solutions to the problems. You are not expected to describe the algorithms used to obtain the solutions.

(a.) (10 points) **Register allocation before instruction scheduling:** Show a register allocation for the original instruction ordering with the minimum number of physical registers. Then, show how the dependence graph changes when physical registers are used instead of virtual registers. Finally, show an instruction schedule with the smallest completion time for this modified dependence graph with physical registers. Report the completion time and number of registers obtained by this approach. Do not use renaming to introduce new registers while performing scheduling.

Minimal Registers:

```
Live ranges:
1. R1 := X + i (t1)
2. R2 := *R1 (t2)
3. *Z := R2
4. R1 := R1 + 2 (t4 overwrites t1)
5. R2 := *N (t5 overwrites t2)
6. R1 := R1 + R2 (t6 overwrites t4)

Register Mapping:
R1: t1 → t4 → t6
R2: t2 → t5
```

Dependence Graph: We observe the same flow edges as provided in the above graph with the following hazards due to reuse:

```
v2 → v4 (0) WAR on R1 (v2 reads R1; v4 writes R1)
v3 → v5 (0) WAR on R2 (v3 reads R2; v5 writes R2)
v1 → v4 (0) WAW on R1 (redundant w/ v1→v4; part of reg graph)
v2 → v5 (0) WAW on R2 (not redundant; no flow)
```

Fastest Completion Time:

```
1: v1
2: v2 (load)
3: v4
4: v3 (needs v2+2)
5: v5 (load)
6: _ (must wait for v5+2)
7: v6
```

We find this is completed, as shown, in 7 cycles with 2 registers.

(b.) (10 points) **Instruction scheduling before register allocation:** Show an instruction schedule with the smallest completion time for the dependence graph above with virtual registers. Then, show a register allocation for this schedule with the minimum number of physical registers. Report the completion time and number of registers obtained by this approach.

Fastest Completion Time:

```
1: v1
2: v2 (load)
3: v5 (load)
```

```
4: v3 (after v2+2)
5: v4
6: v6 (after v4+1 and v5+2)
```

We find this has a completion time of 6 cycles.

Minimal Registers:

```
Live ranges:
After 1: t1
After 2: t1,t2
After 3: t1,t2,t5 // need 3 REG
After 4: t1,t5
After 5: t4,t5
After 6: t6

Register Mapping:
R1: t1 → t4 → t6
R2: t2
R3: t5
```

We find this takes 6 cycles and utilizes 3 registers.

(c.) (10 points) **Combined approach:** Perform both register allocation and instruction scheduling on this example so as to obtain a solution that uses only 2 registers, and has a completion time of 7 cycles. Show your solution.

We begin by using the same allocation as in part (a):

```
R1: t1 → t4 → t6
R2: t2 → t5

Schedule:
1: v1
2: v2 (load)
3: v4
4: v3
5: v5 (load)
6: _
7: v6
```

We observe this utilizes 2 registers and takes 7 cycles.